

AUDITORÍA TÉCNICA COMPLETA - PROYECTO MINIMARKET

RESUMEN EJECUTIVO

Proyecto: Sistema de Gestión de Minimarket

Stack: Node.js, Express, PostgreSQL, Sequelize ORM, EJS

Nivel de Calidad General: 6.5/10

El proyecto muestra una **estructura funcional sólida** con implementación de patrones MVC, autenticación robusta y un sistema de roles bien definido. Sin embargo, presenta **vulnerabilidades de seguridad críticas**, problemas de validación y oportunidades significativas de mejora en arquitectura, rendimiento y experiencia de usuario

1. CALIDAD DEL CÓDIGO BACKEND

FORTALEZAS IDENTIFICADAS

1.1. Estructura Modular Organizada

- Separación clara de responsabilidades: config, models, routes, middleware, views
- Uso apropiado del patrón MVC
- Modelos bien definidos con Sequelize

1.2. Sistema de Autenticación Robusto

```
// middleware/auth.js - Buen manejo de sesiones + JWT híbrido
exports.protect = async (req, res, next) => {
  // Intenta JWT primero, fallback a sesiones
  // Manejo diferenciado de respuestas JSON vs HTML
}
```

1.3. Validaciones de Contraseña Bien Implementadas

```
// models/User.js - Validaciones en hooks
if (password.length < 8) throw new Error('...');
if (!/[A-Z]/.test(password)) throw new Error('...');
// Incluye mayúsculas, minúsculas, números y caracteres especiales
```

1.4. Protección contra Fuerza Bruta

```
// routes/auth.js - Sistema de bloqueo de intentos
const MAX_ATTEMPTS = 5;
const LOCK_TIME_MS = 15 * 60 * 1000; // 15 minutos
```

PROBLEMAS CRÍTICOS DETECTADOS

VULNERABILIDAD #1: SQL INJECTION (CRÍTICO)

Ubicación: routes/api.js líneas 22-43

```
// ✗ CÓDIGO VULNERABLE
where: {
  [Op.and]: [
    {
      [Op.or]: [
        { name: { [Op.iLike]: `%${q}%` } }, // Entrada sin sanitizar
        isNumeric ? { id: parseInt(q, 10) } : null
      ].filter(Boolean)
    },
    sequelize.literal(`(
      SELECT COALESCE(SUM(quantity), 0)
      FROM "Batches"
      WHERE "Batches"."productId" = "Product".id
    ) > 0`)
  ]
}
```

RIESGO: Un atacante puede inyectar caracteres especiales en el parámetro q para manipular la consulta SQL.

SOLUCIÓN:

```
// ✓ CÓDIGO SEGURO
const { Op } = require('sequelize');

router.get('/products/search', protect, cajero, async (req, res) => {
  try {
    const { q } = req.query;

    // Sanitización estricta
    if (!q || typeof q !== 'string') {
      return res.json([]);
    }

    const sanitizedQuery = q.trim().slice(0, 100); // Limitar longitud
    if (!/^[a-zA-Z0-9\sáéíóúñÁÉÍÓÚÑ-]+$/.test(sanitizedQuery)) {
      return res.status(400).json({ message: 'Consulta inválida' });
    }

    const isNumeric = /^d+$/.test(sanitizedQuery);

    const products = await Product.findAll({
      attributes: {
        include: [
          [
            sequelize.fn('COALESCE',
              sequelize.fn('SUM', sequelize.col('batches.quantity')),
              0
            )
          ]
        ]
      }
    });
  } catch (error) {
    // Manejo de errores
  }
}
```

```

        ),
        'totalStock'
    ]
  ],
},
include: [{
  model: Batch,
  as: 'batches',
  attributes: [],
  required: false
}],
where: {
  [Op.or]: [
    { name: { [Op.iLike]: `%${sanitizedQuery}%` } },
    ...(isNumeric ? [{ id: parseInt(sanitizedQuery, 10) }] : [])
  ]
},
group: ['Product.id'],
having: sequelize.where(
  sequelize.fn('COALESCE',
    sequelize.fn('SUM', sequelize.col('batches.quantity')),
    0
  ),
  Op.gt,
  0
),
limit: 10
});

res.json(products);
} catch (error) {
  console.error('Error en búsqueda:', error);
  res.status(500).json({ message: 'Error al buscar productos' });
}
});

```

VULNERABILIDAD #2: XSS (Cross-Site Scripting)

Ubicación: Múltiples vistas EJS

<!-- **X** VULNERABLE -->

<h3>Bienvenido, <%= user.name %></h3>

<p><%= message %></p>

RIESGO: Si user.name o message contienen código JavaScript, se ejecutará en el navegador.

SOLUCIÓN:

<!--  SEGURO - Usar <%- %> solo para HTML confiable -->

<h3>Bienvenido, <%- user.name.replace(/</g, '<').replace(/>/g, '>') %></h3>

<!-- Mejor aún, usar una función helper -->

<h3>Bienvenido, <%= sanitize(user.name) %></h3>

Crear helper en server.js:

```
// Agregar antes de las rutas
app.locals.sanitize = (str) => {
  if (!str) return '';
  return String(str)
    .replace(/&/g, '&amp;')
    .replace(/</g, '&lt;')
    .replace(/>/g, '&gt;')
    .replace(/"/g, '&quot;')
    .replace(/'/g, '&#x27;');
};
```

VULNERABILIDAD #3: CSRF (Cross-Site Request Forgery)

Ubicación: Todas las rutas POST sin protección CSRF

PROBLEMA: No existe protección contra ataques CSRF.

SOLUCIÓN:

```
// Instalar csurf
// npm install csurf

const csrf = require('csrf');
const csrfProtection = csrf({ cookie: true });

// En server.js, después de cookieParser()
app.use(csrfProtection);

// Middleware para pasar token a todas las vistas
app.use((req, res, next) => {
  res.locals.csrfToken = req.csrfToken();
  next();
});

// En formularios EJS
<form method="POST" action="/auth/login">
  <input type="hidden" name="_csrf" value="<%= csrfToken %>">
```

```
<!-- resto del formulario -->
</form>
```

PROBLEMA #4: Manejo Inconsistente de Errores

Ubicación: routes/sales.js línea 74

```
// ✗ EXPOSICIÓN DE STACK TRACES
catch (error) {
  console.error('Error al registrar la venta:', error);
  res.status(500).json({ message: error.message }); // Expone detalles
  internos
}
```

SOLUCIÓN:

```
/ ✓ MANEJO SEGURO
catch (error) {
  console.error('Error al registrar la venta:', error);

  // Log completo en servidor
  logger.error('Sale creation failed', {
    error: error.stack,
    userId: req.session.user.id,
    cart: req.body.cart
  });

  // Mensaje genérico al cliente
  res.status(500).json({
    message: 'Error al procesar la venta. Por favor, intente nuevamente.'
  });
}
```

PROBLEMA #5: Falta de Validación de Entrada

Ubicación: routes/products.js línea 48

```
// ✗ SIN VALIDACIÓN
router.post('/:id/update', protect, admin, upload.single('image'), async
(req, res) => {
  const { id } = req.params;
  const { name, description, price } = req.body;
  // No valida que price sea un número válido
  // No valida longitud de name/description
}
```

SOLUCIÓN:

```
// ✓ CON VALIDACIÓN
const { body, param, validationResult } = require('express-validator');

router.post('/:id/update',
  protect,
```

```

admin,
upload.single('image'),
[
  param('id').isInt({ min: 1 }).toInt(),
  body('name')
    .trim()
    .isLength({ min: 1, max: 200 })
    .escape()
    .withMessage('Nombre inválido'),
  body('description')
    .optional()
    .trim()
    .isLength({ max: 1000 })
    .escape(),
  body('price')
    .isFloat({ min: 0.01, max: 999999 })
    .toFloat()
    .withMessage('Precio inválido')
],
async (req, res) => {
  const errors = validationResult(req);
  if (!errors.isEmpty()) {
    req.session.messages = errors.array().map(e => ({
      type: 'error',
      text: e.msg
    }));
    return res.redirect('/dashboard');
  }
  // Resto del código...
}
);

```

PROBLEMA #6: Falta de Sanitización en Archivos Subidos

Ubicación: middleware/upload.js línea 16

```

// ✗ VALIDACIÓN INSUFICIENTE
const fileFilter = (req, file, cb) => {
  if (file.mimetype.startsWith('image/')) {
    cb(null, true);
  } else {
    cb(new Error('Solo se permiten archivos de imagen'), false);
  }
};

```

RIESGO: Un atacante puede cambiar el mimetype de un archivo malicioso.

SOLUCIÓN:

```
const path = require('path');
```

```

const crypto = require('crypto');

const fileFilter = (req, file, cb) => {
  // Verificar extensión
  const allowedExtensions = ['.jpg', '.jpeg', '.png', '.gif', '.webp'];
  const ext = path.extname(file.originalname).toLowerCase();

  if (!allowedExtensions.includes(ext)) {
    return cb(new Error('Extensión de archivo no permitida'), false);
  }

  // Verificar mimetype
  const allowedMimes = ['image/jpeg', 'image/png', 'image/gif',
'image/webp'];
  if (!allowedMimes.includes(file.mimetype)) {
    return cb(new Error('Tipo MIME no permitido'), false);
  }

  cb(null, true);
};

const storage = multer.diskStorage({
  destination: function (req, file, cb) {
    cb(null, 'public/img/products/');
  },
  filename: function (req, file, cb) {
    // Generar nombre aleatorio seguro
    const randomName = crypto.randomBytes(16).toString('hex');
    const ext = path.extname(file.originalname).toLowerCase();
    cb(null, `product-${randomName}${ext}`);
  }
});

const upload = multer({
  storage: storage,
  fileFilter: fileFilter,
  limits: {
    fileSize: 5 * 1024 * 1024, // 5MB
    files: 1
  }
});

```

PROBLEMA #7: Uso Inconsistente de Transacciones

Ubicación: routes/api.js línea 92

```

// ✗ SIN TRANSACCIÓN
router.post('/products', protect, admin, async (req, res) => {

```

```
const t = await sequelize.transaction();
try {
  const product = await Product.create({...}, { transaction: t });
  await Batch.create({...}, { transaction: t });
  await t.commit();
} catch (error) {
  await t.rollback();
}
});
```

BIEN HECHO: Usa transacciones, pero...

COMPARAR CON: routes/products.js línea 48 - **NO usa transacciones**

RECOMENDACIÓN: Estandarizar el uso de transacciones en **todas** las operaciones que modifican múltiples tablas.

PROBLEMA #8: Exposición de Información Sensible

Ubicación: middleware/auth.js línea 33

```
// ❌ RIESGO DE EXPOSICIÓN
req.session.user = {
  id: user._id, // ¿_id? Debería ser user.id (inconsistencia)
  name: user.name,
  email: user.email, // Email en sesión puede ser innecesario
  role: user.role
};
```

SOLUCIÓN:

```
// ✅ MÍNIMA INFORMACIÓN NECESARIA
req.session.user = {
  id: user.id,
  name: user.name,
  role: user.role
  // Email solo cuando se necesite explícitamente
};
```

2. 🏗️ ARQUITECTURA Y ESTÁNDARES

✅ ASPECTOS POSITIVOS

2.1. Patrón MVC Implementado

- Modelos en /models
- Vistas en /views
- Controladores implícitos en /routes (podría mejorarse)

2.2. Middleware Bien Organizado

- Autenticación centralizada
- Upload manejado por middleware dedicado

2.3. Sistema de Roles Completo

```
// 4 roles: user, admin, inventario, cajero
exports.admin = (req, res, next) => { /* ... */ }
exports.cajero = (req, res, next) => { /* ... */ }
exports.inventario = (req, res, next) => { /* ... */ }
exports.adminOrInventario = (req, res, next) => { /* ... */ }
```

❌ PROBLEMAS DE ARQUITECTURA

PROBLEMA #1: Falta de Capa de Servicios

ACTUAL: Lógica de negocio mezclada en rutas

```
// routes/sales.js - Lógica compleja en la ruta
router.post('/', protect, cajero, async (req, res) => {
  // 60+ líneas de lógica de negocio aquí
  const sale = await Sale.create({...});
  for (const item of cart) {
    // Procesamiento complejo
  }
});
```

RECOMENDACIÓN: Crear capa de servicios

```
// services/SaleService.js
class SaleService {
  async createSale(userId, cart, totalAmount, cashReceived, changeGiven)
  {
    const t = await sequelize.transaction();

    try {
      const sale = await this._createSaleRecord(userId, totalAmount,
cashReceived, changeGiven, t);
      await this._processSaleItems(sale.id, cart, t);
      await this._updateStock(cart, t);

      await t.commit();
      return sale;
    } catch (error) {
      await t.rollback();
      throw error;
    }
  }
}
```

```

    async _createSaleRecord(userId, totalAmount, cashReceived, changeGiven,
transaction) {
        return await Sale.create({
            userId,
            totalAmount,
            cashReceived,
            changeGiven
        }, { transaction });
    }

    async _processSaleItems(saleId, cart, transaction) {
        for (const item of cart) {
            await SaleProduct.create({
                saleId,
                productId: item.id,
                quantity: item.quantity,
                unitPrice: item.price,
                totalPrice: item.quantity * item.price
            }, { transaction });
        }
    }

    async _updateStock(cart, transaction) {
        for (const item of cart) {
            await this._decreaseStockFIFO(item.id, item.quantity, transaction);
        }
    }

    async _decreaseStockFIFO(productId, quantity, transaction) {
        let remaining = quantity;
        const batches = await Batch.findAll({
            where: { productId },
            order: [['purchaseDate', 'ASC'], ['createdAt', 'ASC']],
            transaction
        });

        for (const batch of batches) {
            if (remaining <= 0) break;

            const availableInBatch = batch.quantity;
            if (availableInBatch >= remaining) {
                batch.quantity -= remaining;
                remaining = 0;
            } else {
                remaining -= availableInBatch;
                batch.quantity = 0;
            }
        }
    }

```

```

    }
    await batch.save({ transaction });
  }

  if (remaining > 0) {
    throw new Error(`Stock insuficiente para producto ID:
    ${productId}`);
  }
}
}

module.exports = new SaleService();

// routes/sales.js - SIMPLIFICADO
const SaleService = require('../services/SaleService');

router.post('/', protect, cajero, async (req, res) => {
  const { userId, cart, totalAmount, cashReceived, changeGiven } =
  req.body;

  try {
    const sale = await SaleService.createSale(userId, cart, totalAmount,
    cashReceived, changeGiven);
    res.status(201).json({ message: 'Venta registrada con éxito', saleId:
    sale.id });
  } catch (error) {
    console.error('Error al registrar la venta:', error);
    res.status(500).json({ message: 'Error al procesar la venta' });
  }
});

```

PROBLEMA #2: Falta de Validadores Reutilizables

RECOMENDACIÓN: Crear módulo de validadores

```

// validators/productValidator.js
const { body, param } = require('express-validator');

module.exports = {
  createProduct: [
    body('name')
      .trim()
      .isLength({ min: 1, max: 200 })
      .escape()
      .withMessage('Nombre del producto requerido (máx 200 caracteres)'),
    body('description')
      .optional()
      .trim()

```

```

        .isLength({ max: 1000 })
        .escape(),
    body('price')
        .isFloat({ min: 0.01, max: 999999.99 })
        .toFloat()
        .withMessage('Precio debe ser un número válido mayor a 0'),
    body('stock')
        .optional()
        .isInt({ min: 0 })
        .toInt()
  ],

  updateProduct: [
    param('id').isInt({ min: 1 }).toInt(),
    body('name')
        .trim()
        .isLength({ min: 1, max: 200 })
        .escape(),
    body('description')
        .optional()
        .trim()
        .isLength({ max: 1000 })
        .escape(),
    body('price')
        .isFloat({ min: 0.01, max: 999999.99 })
        .toFloat()
  ]
};

// Uso en routes/products.js
const productValidator = require('../validators/productValidator');

router.post('/', protect, admin, upload.single('image'),
  productValidator.createProduct,
  handleValidationErrors, // Middleware para manejar errores
  async (req, res) => {
    // Código del controlador
  }
);

```

PROBLEMA #3: Configuración No Centralizada

ACTUAL: Variables de entorno dispersas

RECOMENDACIÓN: Centralizar configuración

```

/ config/config.js
require('dotenv').config();

```

```

module.exports = {
  app: {
    port: process.env.PORT || 3000,
    env: process.env.NODE_ENV || 'development'
  },

  db: {
    host: process.env.DB_HOST || 'localhost',
    port: process.env.DB_PORT || 5432,
    name: process.env.DB_NAME,
    user: process.env.DB_USER,
    password: process.env.DB_PASSWORD
  },

  session: {
    secret: process.env.JWT_SECRET,
    maxAge: 30 * 60 * 1000, // 30 minutos
    secure: process.env.NODE_ENV === 'production'
  },

  upload: {
    maxSize: 5 * 1024 * 1024, // 5MB
    allowedExtensions: ['.jpg', '.jpeg', '.png', '.gif', '.webp'],
    allowedMimetypes: ['image/jpeg', 'image/png', 'image/gif',
'image/webp']
  },

  security: {
    maxLoginAttempts: 5,
    lockoutDuration: 15 * 60 * 1000, // 15 minutos
    bcryptRounds: 10
  }
};

// Uso:
const config = require('./config/config');
app.listen(config.app.port, () => {
  console.log(`Server running on port ${config.app.port}`);
});

```

PROBLEMA #4: Falta de Logging Estructurado

ACTUAL: Solo console.log y console.error

RECOMENDACIÓN: Implementar Winston

```
// config/logger.js
```

```

const winston = require('winston');
const path = require('path');

const logger = winston.createLogger({
  level: process.env.LOG_LEVEL || 'info',
  format: winston.format.combine(
    winston.format.timestamp(),
    winston.format.errors({ stack: true }),
    winston.format.json()
  ),
  transports: [
    // Logs de error
    new winston.transports.File({
      filename: path.join(__dirname, '../logs/error.log'),
      level: 'error'
    }),
    // Todos los logs
    new winston.transports.File({
      filename: path.join(__dirname, '../logs/combined.log')
    })
  ]
});

// En desarrollo, también mostrar en consola
if (process.env.NODE_ENV !== 'production') {
  logger.add(new winston.transports.Console({
    format: winston.format.combine(
      winston.format.colorize(),
      winston.format.simple()
    )
  }));
}

module.exports = logger;

// Uso:
const logger = require('./config/logger');

logger.info('Server started', { port: 3000 });
logger.error('Database connection failed', { error: err.message, stack: err.stack });
logger.warn('Low stock detected', { productId: 123, stock: 5 });

```

PROBLEMA #5: Falta de Manejo Centralizado de Errores

RECOMENDACIÓN:

```
// middleware/errorHandler.js
```

```
const logger = require('../config/logger');

class AppError extends Error {
  constructor(message, statusCode) {
    super(message);
    this.statusCode = statusCode;
    this.isOperational = true;
    Error.captureStackTrace(this, this.constructor);
  }
}

const errorHandler = (err, req, res, next) => {
  err.statusCode = err.statusCode || 500;
  err.status = err.status || 'error';

  // Log del error
  logger.error('Error Handler', {
    message: err.message,
    stack: err.stack,
    url: req.originalUrl,
    method: req.method,
    ip: req.ip,
    userId: req.session?.user?.id
  });

  // En producción, no enviar stack trace
  if (process.env.NODE_ENV === 'production') {
    if (err.isOperational) {
      res.status(err.statusCode).json({
        status: err.status,
        message: err.message
      });
    } else {
      res.status(500).json({
        status: 'error',
        message: 'Algo salió mal'
      });
    }
  } else {
    // En desarrollo, enviar toda la información
    res.status(err.statusCode).json({
      status: err.status,
      message: err.message,
      stack: err.stack,
      error: err
    });
  }
};
```

```

    }
  };

module.exports = { AppError, errorHandler };

// En server.js (al final, después de todas las rutas)
const { errorHandler } = require('./middleware/errorHandler');
app.use(errorHandler);

// Uso en rutas:
const { AppError } = require('./middleware/errorHandler');

router.get('/products/:id', async (req, res, next) => {
  try {
    const product = await Product.findByPk(req.params.id);
    if (!product) {
      return next(new AppError('Producto no encontrado', 404));
    }
    res.json(product);
  } catch (error) {
    next(error);
  }
});

```

3. 🎨 UI/UX DE LA APLICACIÓN WEB

✅ ASPECTOS POSITIVOS

3.1. Diseño Responsive Implementado

- Grid system adaptativo: grid-cols-3 sm:grid-cols-4 lg:grid-cols-5
- Breakpoints bien definidos
- Menú móvil funcional

3.2. Paleta de Colores Consistente

```

:root {
  --verde-pastel-1: #b8e6d5;
  --verde-pastel-2: #a0dcc7;
  --verde-pastel-3: #88d4b8;
  --verde-oscuro: #52b788;
  --verde-muy-claro: #e8f5f0;
}

```

3.3. Feedback Visual Adecuado

- Notificaciones toast
- Estados de carga

- Confirmaciones de acciones

✗ PROBLEMAS DE UX/UI

PROBLEMA #1: Accesibilidad Deficiente

Problemas detectados:

1. Falta de etiquetas ARIA

```
2. <!-- ✗ SIN ARIA -->
3. <button onclick="addToCart(...)">
4.   <i class="fas fa-plus"></i>
5. </button>
6.
7. <!-- ☑ CON ARIA -->
8. <button onclick="addToCart(...)"
9.       aria-label="Agregar al carrito"
10.      role="button">
11.   <i class="fas fa-plus" aria-hidden="true"></i>
12.   <span class="sr-only">Agregar al carrito</span>
13.</button>
```

2. Contraste de Color Insuficiente

```
3. /* Verificar con WCAG 2.1 AA:
4.   - Texto normal: mínimo 4.5:1
5.   - Texto grande: mínimo 3:1
6. */
7.
8. /* ✗ BAJO CONTRASTE */
9. .text-subtle {
10.   color: #b2bec3; /* sobre #fafbf8 = 2.1:1 */
11. }
12.
13. /* ☑ MEJOR CONTRASTE */
14. .text-subtle {
15.   color: #636e72; /* sobre #fafbf8 = 4.8:1 */
16. }
```

3. Navegación por teclado limitada

```
4. // Agregar soporte para teclado
5. document.addEventListener('keydown', (e) => {
6.   if (e.key === 'Escape') {
```

```

7.     closeMobileMenu();
8.     closeModal();
9.   }
10.
11.   if (e.key === 'Enter' && e.target.classList.contains('product-
    card')) {
12.     // Agregar al carrito con Enter
13.   }
14.});

```

PROBLEMA #2: Mensajes de Error Poco Claros

ACTUAL:

```

// ✗ MENSAJE GENÉRICO
res.status(500).json({ message: 'Error al crear producto' });

```

RECOMENDACIÓN:

```

// ✓ MENSAJES ESPECÍFICOS Y ÚTILES
if (!product.name) {
  return res.status(400).json({
    message: 'El nombre del producto es obligatorio',
    field: 'name',
    suggestion: 'Por favor, ingresa un nombre para el producto'
  });
}

if (product.price <= 0) {
  return res.status(400).json({
    message: 'El precio debe ser mayor a 0',
    field: 'price',
    currentValue: product.price,
    suggestion: 'Ingresa un precio válido (ejemplo: 10.50)'
  });
}

```

PROBLEMA #3: Flujo de Reservas Confuso

OBSERVACIÓN: El usuario no ve claramente:

- Cuánto tiempo tiene para recoger la reserva
- Si puede modificar una reserva
- Qué pasa si no recoge a tiempo

SOLUCIÓN:

<!-- Agregar timeline visual -->

<div class="reservation-timeline">

<div class="timeline-item completed">

<i class="fas fa-check-circle"></i>

Reserva creada

<small><%= formatDate(cart.createdAt) %></small>

</div>

<div class="timeline-item current">

<i class="fas fa-clock"></i>

Tiempo restante

<strong class="countdown" data-expires="<%= cart.expiresAt %>">

<%= getTimeRemaining(cart.expiresAt) %>

</div>

<div class="timeline-item pending">

<i class="fas fa-store"></i>

Recoge en tienda

<small>Antes del <%= formatDate(cart.expiresAt) %></small>

</div>

</div>

<!-- Agregar advertencias visuales -->

<%= const hoursLeft = getHoursUntilExpiration(cart.expiresAt); %>

<%= if (hoursLeft < 24) { %>

<div class="alert alert-warning">

<i class="fas fa-exclamation-triangle"></i>

¡Atención! Tu reserva expira en menos de 24 horas.

Recógela pronto o se cancelará automáticamente.

</div>

<% } %>

PROBLEMA #4: Falta de Estados de Carga

PROBLEMA: Al hacer operaciones lentas (guardar reserva, crear venta), no hay feedback visual.

```
// Agregar spinner/loader
function saveReservation() {
  const btn = document.getElementById('saveReservationBtn');
  btn.disabled = true;
  btn.innerHTML = '<i class="fas fa-spinner fa-spin"></i> Guardando...';

  fetch('/cart/update', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ items: cart })
  })
  .then(response => response.json())
  .then(data => {
    if (data.success) {
      showNotification(`Reserva guardada:
${data.cart.reservationNumber}`, 'success');
      clearCart();
    } else {
      showNotification(data.message, 'error');
    }
  })
  .catch(error => {
    showNotification('Error de conexión. Intenta nuevamente.', 'error');
  })
  .finally(() => {
    btn.disabled = false;
    btn.innerHTML = '<i class="fas fa-bookmark"></i> Guardar Reserva';
  });
}
```

PROBLEMA #5: Tarjetas de Producto Muy Pequeñas en Mobile

OBSERVACIÓN: En el código actual:

<div class="grid grid-cols-3 sm:grid-cols-4 lg:grid-cols-5">

3 columnas en móvil puede ser excesivo si las tarjetas son muy pequeñas.

```

<!-- Mejor legibilidad en móvil -->

<div class="grid grid-cols-2 sm:grid-cols-3 md:grid-cols-4 lg:grid-cols-5 xl:grid-
cols-6">

  <div class="product-card">

    <!-- Imagen más grande en móvil -->

    <div class="product-image" style="padding-bottom: 100%;"> <!-- Ratio 1:1 -->

      

    </div>

    <!-- Texto más legible -->

    <h3 class="text-sm sm:text-xs font-bold truncate"><%= product.name %></h3>

    <p class="text-base sm:text-sm font-bold text-green-600">Bs <%=
product.price.toFixed(2) %></p>

    <!-- Botón táctil más grande en móvil -->

    <button class="w-full py-3 sm:py-2 text-sm sm:text-xs">

      <i class="fas fa-plus"></i>

      <span class="hidden sm:inline">Agregar</span>

    </button>

  </div>

</div>

```

PROBLEMA #6: Falta de Búsqueda Avanzada

RECOMENDACIÓN:

```

<!-- Agregar filtros avanzados -->

<div class="search-filters">

  <input type="text" id="searchInput" placeholder="Buscar productos...">

  <select id="categoryFilter">

    <option value="">Todas las categorías</option>

```

```
<option value="Bebidas">Bebidas</option>
<option value="Lácteos">Lácteos</option>
<!-- ... -->
</select>
```

```
<select id="priceFilter">
  <option value="">Cualquier precio</option>
  <option value="0-10">Menos de Bs 10</option>
  <option value="10-50">Bs 10 - 50</option>
  <option value="50+">Más de Bs 50</option>
</select>
```

```
<select id="stockFilter">
  <option value="">Todos</option>
  <option value="in-stock">En stock</option>
  <option value="low-stock">Stock bajo</option>
</select>
```

```
<select id="sortBy">
  <option value="name-asc">Nombre (A-Z)</option>
  <option value="name-desc">Nombre (Z-A)</option>
  <option value="price-asc">Precio (menor a mayor)</option>
  <option value="price-desc">Precio (mayor a menor)</option>
</select>
```

```
</div>
```

```
<script>
```

```
function filterProducts() {
```

```
const search = document.getElementById('searchInput').value.toLowerCase();
const category = document.getElementById('categoryFilter').value;
const priceRange = document.getElementById('priceFilter').value;
const stockStatus = document.getElementById('stockFilter').value;
const sortBy = document.getElementById('sortBy').value;
```

```
let products = Array.from(document.querySelectorAll('.product-card'));
```

```
// Filtrar
```

```
products = products.filter(card => {
  const name = card.dataset.productName.toLowerCase();
  const cat = card.dataset.productCategory;
  const price = parseFloat(card.dataset.productPrice);
  const stock = parseInt(card.dataset.productStock);
```

```
// Búsqueda de texto
```

```
if (search && !name.includes(search)) return false;
```

```
// Categoría
```

```
if (category && cat !== category) return false;
```

```
// Rango de precio
```

```
if (priceRange) {
  const [min, max] = priceRange.split('-').map(Number);
  if (max) {
    if (price < min || price > max) return false;
  } else {
    if (price < min) return false;
```

```
}  
}
```

```
// Estado de stock
```

```
if (stockStatus === 'in-stock' && stock === 0) return false;
```

```
if (stockStatus === 'low-stock' && (stock === 0 || stock > 10)) return false;
```

```
return true;
```

```
});
```

```
// Ordenar
```

```
products.sort((a, b) => {
```

```
  const aName = a.dataset.productName;
```

```
  const bName = b.dataset.productName;
```

```
  const aPrice = parseFloat(a.dataset.productPrice);
```

```
  const bPrice = parseFloat(b.dataset.productPrice);
```

```
  switch(sortBy) {
```

```
    case 'name-asc': return aName.localeCompare(bName);
```

```
    case 'name-desc': return bName.localeCompare(aName);
```

```
    case 'price-asc': return aPrice - bPrice;
```

```
    case 'price-desc': return bPrice - aPrice;
```

```
    default: return 0;
```

```
  }
```

```
});
```

```
// Renderizar
```

```
const container = document.getElementById('productsContainer');
```



```

container.innerHTML = "";

products.forEach(card => container.appendChild(card));

// Mostrar mensaje si no hay resultados
if (products.length === 0) {
  container.innerHTML = `
    <div class="col-span-full text-center py-8">
      <i class="fas fa-search text-4xl text-gray-400 mb-2"></i>
      <p class="text-gray-600">No se encontraron productos</p>
    </div>
  `;
}
}

// Escuchar cambios en los filtros
document.getElementById('searchInput').addEventListener('input', filterProducts);
document.getElementById('categoryFilter').addEventListener('change',
filterProducts);
document.getElementById('priceFilter').addEventListener('change', filterProducts);
document.getElementById('stockFilter').addEventListener('change',
filterProducts);
document.getElementById('sortBy').addEventListener('change', filterProducts);
</script>

```

4. SEGURIDAD

MEDIDAS DE SEGURIDAD IMPLEMENTADAS

1. **Hashing de Contraseñas con bcrypt**
2. **Protección contra Fuerza Bruta** (lockout después de 5 intentos)
3. **Validación de Contraseñas Robustas**

4. Sesiones Seguras con httpOnly
5. Cookies seguras en producción

✗ VULNERABILIDADES CRÍTICAS

Además de las mencionadas en la sección 1 (SQL Injection, XSS, CSRF), se detectaron:

VULNERABILIDAD #4: Headers de Seguridad Faltantes

SOLUCIÓN:

```
// npm install helmet
const helmet = require('helmet');

app.use(helmet({
  contentSecurityPolicy: {
    directives: {
      defaultSrc: ['self'],
      styleSrc: ['self', "'unsafe-inline'",
        "https://cdn.tailwindcss.com", "https://cdnjs.cloudflare.com"],
      scriptSrc: ['self', "'unsafe-inline'",
        "https://cdn.tailwindcss.com", "https://cdn.jsdelivr.net"],
      imgSrc: ['self', "data:", "https:"],
      fontSrc: ['self', "https://cdnjs.cloudflare.com"],
    }
  },
  hsts: {
    maxAge: 31536000,
    includeSubDomains: true,
    preload: true
  }
}));

// Prevenir clickjacking
app.use(helmet.frameguard({ action: 'deny' }));

// Prevenir MIME sniffing
app.use(helmet.noSniff());

// Deshabilitar X-Powered-By
app.disable('x-powered-by');
```

VULNERABILIDAD #5: Rate Limiting Ausente

SOLUCIÓN

```
// npm install express-rate-limit
const rateLimit = require('express-rate-limit');

// Limitar intentos de login
const loginLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutos
  max: 5, // 5 intentos
  message: 'Demasiados intentos de inicio de sesión. Intenta nuevamente en 15 minutos.',
  standardHeaders: true,
  legacyHeaders: false
});

app.use('/auth/login', loginLimiter);

// Limitar API requests
const apiLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 100,
  message: 'Demasiadas peticiones desde esta IP'
});

app.use('/api/', apiLimiter);

// Limitar creación de recursos
const createLimiter = rateLimit({
  windowMs: 60 * 60 * 1000, // 1 hora
  max: 20,
  message: 'Has alcanzado el límite de creación. Intenta en 1 hora.'
});

app.use('/products', createLimiter);
```

VULNERABILIDAD #6: Sesiones Sin Regeneración

PROBLEMA: Después del login, no se regenera el ID de sesión (vulnerable a session fixation).

SOLUCIÓN:

```
// routes/auth.js
router.post('/login', async (req, res) => {
  const { email, password } = req.body;

  try {
    const user = await User.findOne({ where: { email } });

    if (user && (await user.matchPassword(password))) {
```

```

// ☒ REGENERAR SESIÓN
req.session.regenerate((err) => {
  if (err) {
    console.error('Error regenerating session:', err);
    return res.redirect('/auth/login?m=' +
      encodeURIComponent('Error al iniciar sesión'));
  }

  req.session.user = {
    id: user.id,
    name: user.name,
    role: user.role
  };

  req.session.save((err) => {
    if (err) {
      console.error('Error saving session:', err);
    }
    res.redirect('/dashboard');
  });
});

// Resetear intentos de login
if (user.loginAttempts > 0 || user.lockUntil) {
  user.loginAttempts = 0;
  user.lockUntil = null;
  await user.save();
}
} else {
  // Manejo de error...
}
} catch (err) {
  // ...
}
});

```

VULNERABILIDAD #7: Falta de Validación de Tipos de Archivo en Servidor

Ya mencionado en la sección 1, pero crítico para seguridad.

AGREGAR ADEMÁS:

```

// Escaneo de virus/malware (opcional pero recomendado)
// npm install clamscan

const NodeClam = require('clamscan');

const clamscan = new NodeClam().init({

```

```

    clamscan: {
      host: 'localhost',
      port: 3310
    }
  });

  // Middleware para escanear archivos
  const scanFile = async (req, res, next) => {
    if (!req.file) return next();

    try {
      const { isInfected, viruses } = await
        clamscan.isInfected(req.file.path);

      if (isInfected) {
        // Eliminar archivo infectado
        fs.unlinkSync(req.file.path);
        return res.status(400).json({
          message: 'Archivo rechazado: posible virus detectado',
          viruses
        });
      }

      next();
    } catch (err) {
      console.error('Error scanning file:', err);
      next(); // Continuar aunque falle el escaneo
    }
  };

  router.post('/', protect, admin, upload.single('image'), scanFile, async
    (req, res) => {
      // ...
    });

```

VULNERABILIDAD #8: Información Sensible en Logs

PROBLEMA:

console.error('Error al registrar la venta:', error);

console.log('User data:', user); // Puede incluir contraseña hasheada

SOLUCIÓN:

```

// Sanitizar datos antes de loguear
const sanitizeForLog = (obj) => {
  const sanitized = { ...obj };
  const sensitiveFields = ['password', 'token', 'creditCard', 'ssn'];

```

```

sensitiveFields.forEach(field => {
  if (sanitized[field]) {
    sanitized[field] = '[REDACTED]';
  }
});

return sanitized;
});

logger.info('User logged in', sanitizeForLog(user));

```

5. 📖 BASE DE DATOS POSTGRESQL

✅ ASPECTOS POSITIVOS

1. **Uso de Sequelize ORM** - Previene SQL injection básica
2. **Relaciones bien definidas** - Asociaciones Many-to-Many, One-to-Many correctas
3. **Uso de transacciones** - En operaciones críticas como ventas
4. **Sistema FIFO implementado** - Para gestión de lotes

❌ PROBLEMAS DE BASE DE DATOS

PROBLEMA #1: Falta de Índices Importantes

ANÁLISIS: Índices faltantes pueden causar consultas lentas.

SOLUCIÓN:

```

// models/Product.js
Product.init({
  // ... campos existentes
}, {
  sequelize,
  modelName: 'Product',
  indexes: [
    {
      name: 'idx_product_name',
      fields: ['name']
    },
    {
      name: 'idx_product_category',
      fields: ['category']
    },
  ],
});

```

```

    {
      name: 'idx_product_price',
      fields: ['price']
    },
    // Índice compuesto para búsquedas frecuentes
    {
      name: 'idx_product_category_price',
      fields: ['category', 'price']
    }
  ]
});

```

```

// models/Sale.js
Sale.init({
  // ... campos existentes
}, {
  sequelize,
  modelName: 'Sale',
  indexes: [
    {
      name: 'idx_sale_user',
      fields: ['userId']
    },
    {
      name: 'idx_sale_created',
      fields: ['createdAt']
    },
    // Índice compuesto para reportes
    {
      name: 'idx_sale_user_created',
      fields: ['userId', 'createdAt']
    }
  ]
});

```

```

// models/Batch.js
Batch.init({
  // ... campos existentes
}, {
  sequelize,
  modelName: 'Batch',
  indexes: [
    {
      name: 'idx_batch_product',
      fields: ['productId']
    },
  ],
});

```

```

    {
      name: 'idx_batch_purchase_date',
      fields: ['purchaseDate']
    },
    // Índice para FIFO
    {
      name: 'idx_batch_fifo',
      fields: ['productId', 'purchaseDate', 'createdAt']
    }
  ]
});

// models/Cart.js
Cart.init({
  // ... campos existentes
}, {
  sequelize,
  modelName: 'Cart',
  indexes: [
    {
      name: 'idx_cart_user_status',
      fields: ['userId', 'status']
    },
    {
      name: 'idx_cart_reservation_number',
      fields: ['reservationNumber'],
      unique: true
    },
    {
      name: 'idx_cart_expires_at',
      fields: ['expiresAt']
    }
  ]
});

```

PROBLEMA #2: Falta de Constraints en Base de Datos

PROBLEMA: Algunas validaciones solo están en el código, no en la BD.

SOLUCIÓN:

```

// models/Product.js
Product.init({
  id: {
    type: DataTypes.INTEGER,
    primaryKey: true,
    autoIncrement: true
  },

```



```

name: {
  type: DataTypes.STRING(200), // Longitud específica
  allowNull: false,
  validate: {
    notEmpty: true,
    len: [1, 200]
  }
},
description: {
  type: DataTypes.TEXT,
  allowNull: true
},
price: {
  type: DataTypes.DECIMAL(10, 2), // Mejor que FLOAT para dinero
  allowNull: false,
  validate: {
    min: 0.01,
    max: 999999.99
  }
},
image: {
  type: DataTypes.STRING(500),
  allowNull: true,
  validate: {
    isUrl: true
  }
},
category: {
  type: DataTypes.ENUM('Bebidas', 'Lácteos', 'Panadería', 'Snacks',
'Limpieza', 'Higiene', 'Enlatados', 'Despensa', 'General'),
  defaultValue: 'General',
  allowNull: false
}
}, {
  sequelize,
  modelName: 'Product',
  tableName: 'Products'
});

// models/Sale.js
Sale.init({
  id: {
    type: DataTypes.INTEGER,
    primaryKey: true,
    autoIncrement: true
  },

```

```

    userId: {
      type: DataTypes.INTEGER,
      allowNull: false,
      references: {
        model: 'Users',
        key: 'id'
      },
      onUpdate: 'CASCADE',
      onDelete: 'RESTRICT' // No permitir eliminar usuario con ventas
    },
    totalAmount: {
      type: DataTypes.DECIMAL(10, 2),
      allowNull: false,
      validate: {
        min: 0
      }
    },
    cashReceived: {
      type: DataTypes.DECIMAL(10, 2),
      allowNull: false,
      validate: {
        min: 0,
        // Validar que sea >= totalAmount
        isGreaterOrEqualTotal(value) {
          if (value < this.totalAmount) {
            throw new Error('El efectivo recibido debe ser mayor o igual al
total');
          }
        }
      }
    },
    changeGiven: {
      type: DataTypes.DECIMAL(10, 2),
      allowNull: false,
      validate: {
        min: 0
      }
    }
  }, {
    sequelize,
    modelName: 'Sale',
    timestamps: true
  });
});

// models/Batch.js
Batch.init({

```

```

id: {
  type: DataTypes.INTEGER,
  primaryKey: true,
  autoIncrement: true
},
quantity: {
  type: DataTypes.INTEGER,
  allowNull: false,
  validate: {
    min: 0
  }
},
purchasePrice: {
  type: DataTypes.DECIMAL(10, 2),
  allowNull: false,
  validate: {
    min: 0
  }
},
purchaseDate: {
  type: DataTypes.DATE,
  allowNull: false,
  defaultValue: DataTypes.NOW
},
productId: {
  type: DataTypes.INTEGER,
  allowNull: false,
  references: {
    model: 'Products',
    key: 'id'
  },
  onUpdate: 'CASCADE',
  onDelete: 'CASCADE' // Si se elimina producto, eliminar sus lotes
}, {
  sequelize,
  modelName: 'Batch'
});

```

PROBLEMA #3: Falta de Auditoría / Logs de Cambios

RECOMENDACIÓN: Implementar tabla de auditoría

```

// models/AuditLog.js
const { Model, DataTypes } = require('sequelize');

class AuditLog extends Model {}

```

```

module.exports = (sequelize) => {
  AuditLog.init({
    id: {
      type: DataTypes.INTEGER,
      primaryKey: true,
      autoIncrement: true
    },
    userId: {
      type: DataTypes.INTEGER,
      allowNull: true,
      references: {
        model: 'Users',
        key: 'id'
      }
    },
    action: {
      type: DataTypes.ENUM('CREATE', 'UPDATE', 'DELETE', 'LOGIN',
'LOGOUT'),
      allowNull: false
    },
    tableName: {
      type: DataTypes.STRING(100),
      allowNull: false
    },
    recordId: {
      type: DataTypes.INTEGER,
      allowNull: true
    },
    oldValues: {
      type: DataTypes.JSONB,
      allowNull: true
    },
    newValues: {
      type: DataTypes.JSONB,
      allowNull: true
    },
    ip: {
      type: DataTypes.STRING(45), // IPv6
      allowNull: true
    },
    userAgent: {
      type: DataTypes.TEXT,
      allowNull: true
    }
  }, {
    sequelize,

```

```

    modelName: 'AuditLog',
    tableName: 'AuditLogs',
    timestamps: true,
    updatedAt: false // Solo createdAt
  });

  return AuditLog;
};

// Hook para auditar cambios en Product
Product.addHook('afterCreate', async (product, options) => {
  await AuditLog.create({
    userId: options.userId, // Pasar desde el controlador
    action: 'CREATE',
    tableName: 'Products',
    recordId: product.id,
    newValues: product.toJSON()
  });
});

Product.addHook('afterUpdate', async (product, options) => {
  await AuditLog.create({
    userId: options.userId,
    action: 'UPDATE',
    tableName: 'Products',
    recordId: product.id,
    oldValues: product._previousDataValues,
    newValues: product.dataValues
  });
});

Product.addHook('afterDestroy', async (product, options) => {
  await AuditLog.create({
    userId: options.userId,
    action: 'DELETE',
    tableName: 'Products',
    recordId: product.id,
    oldValues: product.toJSON()
  });
});

// Uso en rutas:
await Product.create({...}, {
  transaction: t,
  userId: req.session.user.id // Pasar userId para auditoría
});

```

PROBLEMA #4: No hay Soft Deletes

RECOMENDACIÓN:

```
// Habilitar paranoid en modelos críticos
Product.init({
  // ... campos existentes
}, {
  sequelize,
  modelName: 'Product',
  paranoid: true, // Activa soft delete
  deletedAt: 'deletedAt' // Nombre de la columna
});

// Ahora Product.destroy() solo marcará como eliminado
await Product.destroy({ where: { id: 123 } });

// Para eliminar permanentemente:
await Product.destroy({ where: { id: 123 }, force: true });

// Para incluir eliminados en consultas:
await Product.findAll({ paranoid: false });

// Para restaurar:
await Product.restore({ where: { id: 123 } });
```

PROBLEMA #5: Falta de Validación de Integridad Referencial

PROBLEMA: Al eliminar un producto que tiene ventas asociadas.

SOLUCIÓN: Ya implementado parcialmente con onDelete: 'RESTRICT', pero falta manejo explícito:

```
router.post('/:id/delete', protect, admin, async (req, res) => {
  try {
    const { id } = req.params;

    // Verificar si tiene ventas asociadas
    const salesCount = await SaleProduct.count({ where: { productId: id } });

    if (salesCount > 0) {
      return res.status(400).json({
        success: false,
        message: 'No se puede eliminar el producto porque tiene ventas asociadas. Considere desactivarlo en su lugar.'
      });
    }

    // Verificar si tiene lotes con stock
```

```

    const batchesWithStock = await Batch.sum('quantity', { where: {
productId: id } });
    if (batchesWithStock > 0) {
      return res.status(400).json({
        success: false,
        message: `No se puede eliminar el producto porque tiene
${batchesWithStock} unidades en stock.`
      });
    }

    await Product.destroy({ where: { id } });
    res.json({ success: true, message: 'Producto eliminado exitosamente.'
});
  } catch (err) {
    console.error(err);
    res.status(500).json({ success: false, message: 'Error al eliminar el
producto.' });
  }
});

```

PROBLEMA #6: Query N+1 en Algunas Rutas

PROBLEMA DETECTADO: En routes/index.js:

```

// ✗ PUEDE CAUSAR N+1
router.get('/dashboard/user', protect, async (req, res) => {
  const products = await Product.findAll({
    include: 'batches'
  });
  // Si hay 100 productos, hace 1 query para productos + 1 query por cada
producto para sus batches
});

```

SOLUCIÓN: Usar required: false y separate: false apropiadamente:

```

// ✔ OPTIMIZADO
router.get('/dashboard/user', protect, async (req, res) => {
  const products = await Product.findAll({
    include: [{
      model: Batch,
      as: 'batches',
      attributes: ['id', 'quantity', 'purchasePrice'], // Solo los campos
necesarios
      required: false,
      separate: false // Evita consultas adicionales
    }],
    order: [['name', 'ASC']]
  });
});

```

```
res.render('dashboard_user', {
  user: req.session.user,
  products,
  search: ''
});
});
```

PROBLEMA #7: Falta de Respaldos Automatizados

RECOMENDACIÓN: Script de backup

```
// scripts/backup-db.js
const { exec } = require('child_process');
const fs = require('fs');
const path = require('path');
require('dotenv').config();

const BACKUP_DIR = path.join(__dirname, '../backups');

// Crear directorio si no existe
if (!fs.existsSync(BACKUP_DIR)) {
  fs.mkdirSync(BACKUP_DIR, { recursive: true });
}

const timestamp = new Date().toISOString().replace(/[:.]/g, '-');
const filename = `backup-${timestamp}.sql`;
const filepath = path.join(BACKUP_DIR, filename);

const command = `pg_dump -U ${process.env.DB_USER} -h
${process.env.DB_HOST} -p ${process.env.DB_PORT} ${process.env.DB_NAME} >
${filepath}`;

exec(command, { env: { PGPASSWORD: process.env.DB_PASSWORD } }, (error,
stdout, stderr) => {
  if (error) {
    console.error('Error al crear backup:', error);
    return;
  }

  console.log(`Backup creado exitosamente: ${filename}`);

  // Eliminar backups antiguos (mantener solo los últimos 7 días)
  const files = fs.readdirSync(BACKUP_DIR);
  const sevenDaysAgo = Date.now() - (7 * 24 * 60 * 60 * 1000);

  files.forEach(file => {
    const filePath = path.join(BACKUP_DIR, file);
    const stats = fs.statSync(filePath);
```



```

    if (stats.mtimeMs < sevenDaysAgo) {
      fs.unlinkSync(filePath);
      console.log(`Backup antiguo eliminado: ${file}`);
    }
  });
});

// Agendar con cron (agregar en package.json)
// "scripts": {
//   "backup": "node scripts/backup-db.js"
// }

// Configurar cron job (Linux/Mac) o Task Scheduler (Windows)
// 0 2 * * * cd /path/to/project && npm run backup

```

6. ⚡ RENDIMIENTO

✖ PROBLEMAS DE RENDIMIENTO

PROBLEMA #1: Falta de Caché

RECOMENDACIÓN: Implementar Redis para caché

```

// npm install redis
const redis = require('redis');
const client = redis.createClient({
  host: process.env.REDIS_HOST || 'localhost',
  port: process.env.REDIS_PORT || 6
});

```